LANGSMITH VS. LANGFUSE CHEATSHEET

AI Engineer Ready · Week 6 — RAG Optimization + Evaluation + LangSmith

Golden Rule -> same job, different company -- pick LangSmith for LangChain-native ease, Langfuse for self-hosting

1. What Is Langfuse?

	Detail
Open-source core	The core tracing and eval engine is open-source -- you can read the code, self-host it, or use their cloud.
Framework-agnostic	Works with LangChain, LlamaIndex, or plain API calls -- it doesn't assume you're using any particular framework.
Tracing + evals + prompts	Covers the same ground as LangSmith: traces, datasets, evaluators, and prompt management.

Langfuse being open-source doesn't just mean it's free **--** it means a company can run it entirely inside their own infrastructure, which matters a lot for regulated industries like insurance or healthcare.

2. LangSmith vs. Langfuse, Head to Head

	LangSmith	Langfuse
Built by	The LangChain team	Independent open-source project
Framework fit	Deepest native fit for LangChain / LangGraph	Framework-agnostic -- LangChain, LlamaIndex, raw API calls
Hosting	Managed SaaS, with a free tier	Open-source core -- self-host or use their cloud
Pricing	Scales with trace volume	Free if self-hosted; usage-based cloud tier
Standout feature	Strong Prompt Hub for shared prompts	Strong cost and latency dashboards
Eval features	Mature datasets, evaluators, compare runs	Growing eval + prompt management features

3. Rule of Thumb

If you...	Pick...
Are all-in on LangChain and want the path of least resistance	LangSmith
Need self-hosting, data residency, or are framework-agnostic	Langfuse

For a regulated industry **--** banking, insurance, healthcare **--** the ability to self-host Langfuse entirely inside your own servers can be the deciding factor, independent of which tool has more features.

4. Quick Setup

LANGSMITH VS. LANGFUSE CHEATSHEET

AI Engineer Ready · Week 6 — RAG Optimization + Evaluation + LangSmith

Golden Rule -> same job, different company -- pick LangSmith for LangChain-native ease, Langfuse for self-hosting

```
$ # LangSmith -- environment variables, then it traces automatically
$ export LANGCHAIN_TRACING_V2=true
$ export LANGCHAIN_API_KEY="your-langsmith-api-key"
$ export LANGCHAIN_PROJECT="my-project"

$ # Langfuse -- wrap your callable with the Langfuse handler
$ from langfuse.callback import CallbackHandler

$ langfuse_handler = CallbackHandler(
$ public_key="pk-...", secret_key="sk-...",
$ )
$ chain.invoke(question, config={"callbacks": [langfuse_handler]})
```

Pitfall

More tools means more setup and upkeep. Adopting both LangSmith and Langfuse on the same project without a clear reason just doubles your configuration and maintenance burden for no extra insight.

Remember

LangSmith and Langfuse are asking the same question -- what did every LLM call actually do -- just from two very different companies and hosting models. Pick based on your framework and hosting constraints, not feature-count alone.

INTERVIEW PREP

Q: Which observability tool have you used, and why would you pick one over the other?

A: LangSmith gives the smoothest experience when you're building entirely in LangChain/LangGraph and want a managed SaaS with minimal setup. Langfuse is the better call when you need self-hosting, data residency, or you're not committed to one framework.

Q: Why would a regulated company (bank, insurer, hospital) prefer Langfuse?

A: Because its open-source core can be self-hosted entirely inside the company's own infrastructure, keeping sensitive trace data from ever leaving their servers -- something a purely managed SaaS tool can't offer.

Q: Do LangSmith and Langfuse cover fundamentally different capabilities?

A: No -- both cover tracing, datasets, evaluators, and prompt management. The real differentiator is company/hosting model and framework fit, not the underlying feature set.